PERFORMANCE ENGINEERING · TOOLING RECOMMENDATION

k6 vs JMeter

A performance-testing platform recommendation for Softaims — supported by a benchmark we ran on our own hardware.

Prepared for engineering leadership September 2026 k6 v2.2.0 · JMeter 5.6.3

THE RECOMMENDATION

Adopt k6 as the default. Keep JMeter where it still wins.

STRATEGIC DEFAULT

k6

Every new performance-testing effort: REST, GraphQL, gRPC, WebSocket, microservices, Kubernetes, and CI performance gates.

- Tests are code — reviewed in Git like everything else
- Pass/fail thresholds built in, so CI can block a bad build
- Arrival-rate models that reflect real customer behaviour
- Far lighter on the load-generator machine

SPECIALISED / LEGACY

JMeter

Retained deliberately — not deprecated — for the workloads k6 genuinely cannot reach.

- JDBC, JMS, LDAP, FTP, SMTP and other enterprise protocols
- Existing, stable `.jmx` suites that already deliver value
- GUI-first workflows for QA colleagues who do not write code
- Two decades of institutional knowledge and plugins

WHY NOT SIMPLY REPLACE JMETER

Softaims already advertises JMeter capability and holds real expertise in it. Removing it would discard an asset to solve a problem we do not have. The strongest position is not “k6 *or* JMeter” — it is **k6 as the strategic platform, JMeter as the specialist tool.**

ASSESSMENT

Scored against how Softaims actually works

AREA	K6	JMETER
Measurement & workload accuracy	9.2	8.5
CPU / resource efficiency	9.5	6.5
CI/CD integration	10	7.5
Maintainability over years	9.5	7.0
Protocol breadth	7.5	10
Initial GUI usability	8.0	9.0
Observability	9.5	7.5
Enterprise maturity	9.0	10
Modern development workflow	10	7.0
Weighted for Softaims	**8.9**	**7.9**

JMeter wins three categories

Protocol breadth, GUI usability and enterprise maturity. Each is real — and none of them is on the critical path for the API and cloud work that dominates our pipeline.

READ THIS HONESTLY

These weights are an **engineering judgement** about Softaims' workload mix, not a measurement. A company running mostly JDBC and JMS testing would score the same two tools the other way round.

ROOT CAUSE

One architectural decision explains most of the difference

JMETER · JVM

```
Virtual User
     ↓
Java Thread          ← one OS thread each
     ↓
HTTP / JDBC / JMS / …
```

Each virtual user is backed by a real operating-system thread. Simple to reason about, but the machine pays for every one of them in memory and scheduling.

K6 · GO

```
Virtual User
     ↓
lightweight context   ← multiplexed
     ↓
Go runtime
     ↓
HTTP / gRPC / WebSocket / …
```

Many virtual users share a small pool of OS threads. Test logic is JavaScript; the engine underneath is compiled Go.

MEASURED ON OUR OWN MACHINE — 1,000 VIRTUAL USERS

JMeter created **1,037 operating-system threads**. k6 used **15**. That is not a tuning setting — it is the architecture, and it drives the resource findings on the next slides.

ACCURACY

The real risk is not arithmetic — it is coordinated omission

Both tools compute averages, percentiles, throughput and error rates correctly. The question that actually matters is whether the generator delivered the workload you asked for.

Closed model — the trap

```
Target        1,000 RPS

Server slows down:
200ms → 500ms → 1s → 3s

Actual load delivered:
1000 → 850 → 600 → 300
```

Users wait for a response before asking again, so the test **withdraws pressure exactly when the system is struggling** — and reports a healthier result than reality.

Open model — the fix

```
constant-arrival-rate
ramping-arrival-rate

Requests start on a clock,
regardless of whether earlier
ones have returned.

Load stays at 1,000 RPS
```

This is how real customers behave: they arrive when they arrive. k6 also reports `dropped_iterations` when it *cannot* keep up — telling you the generator, not the server, is the limit.

STATED FAIRLY

JMeter can model open workloads too, via its **Open Model Thread Group** and Precise Throughput Timer. This is not something JMeter cannot do — the semantics are simply cleaner and harder to misuse in k6. Apache's own documentation warns that incorrect thread sizing causes coordinated omission.

MEASUREMENT TRAP

Do not expect the two tools to report the same number

k6 · http_req_duration

```
  sending
+ waiting
+ receiving
─────────────
  http_req_duration

Reported separately:
  http_req_connecting
  http_req_tls_handshaking
```

DNS, TCP and TLS setup are **excluded** from the headline duration and exposed as their own metrics.

JMeter · Elapsed Time

```
From just before the request
is sent, until the complete
response has been received.

Reported separately:
  Latency
  Connect Time
```

A different span, measured against a different starting point.

THE CONSEQUENCE

`k6 = 240 ms` and `JMeter = 255 ms` does **not** mean JMeter is slower or less accurate. Before comparing any two tools, confirm you are comparing equivalent metric definitions — otherwise you are measuring the documentation, not the system.

EVIDENCE · MEASURED IN-HOUSE

We ran both tools ourselves rather than citing a blog

Method

Identical load profiles against a local target with a **known, fixed 50 ms latency** — so whatever each tool *reports* can be checked against the truth. 1,000 virtual users, 30 s ramp, 60 s hold. Load-generator process sampled every 500 ms.

Environment

Windows 11 · 8 logical cores · 15.6 GB RAM
k6 v2.1.0 · JMeter 5.6.3 · OpenJDK 17
No listeners attached · both runs 0% errors

METRIC	JMETER	K6
Peak memory	841 MB	320 MB
Peak OS threads	1,037	15
CPU seconds	28.19	21.08
Memory per VU	0.76 MB	0.17 MB
Requests completed	69,027	71,343
Throughput	765/s	784/s
Reported latency true value = 50 ms	61.7 ms	56.8 ms

WHAT THIS EARNS US

Every figure is reproducible from the harness in our repository. In a room, “we measured this on our hardware” ends an argument that “a blog said so” only starts.

RESOURCE CONSUMPTION

k6 is lighter — and the gap widens with scale

PEAK MEMORY

2.6×

less than JMeter — measured at 1,000 VUs

OS THREADS

69×

fewer — 15 against 1,037

CPU CONSUMED

1.34×

less for equivalent work

AN HONEST CAVEAT ON THE BIGGER PUBLISHED NUMBERS

A widely cited 2026 benchmark reports far larger gaps at **10,000** VUs — 2.8 GB against 22.4 GB, and roughly 3.3× the VU density per node. Our 1,000-VU result shows a more modest 2.6×. Both can be true: **the thread-per-user cost compounds as concurrency rises.** Treat published figures as directional, and size infrastructure from our own measurements.

OPERATING DISCIPLINE

Most JMeter overhead complaints are configuration, not the tool

What distorts a JMeter test

```
JMeter GUI
+ View Results Tree
+ View Results Table
+ assertions everywhere
+ multiple plugins
+ large saved responses
```

Apache's own best-practice guidance names these directly: run in non-GUI mode, avoid heavy listeners during the run, minimise assertions and limit saved response data.

How production JMeter should run

```
jmeter -n -t test.jmx -l results.jtl

then, separately:

jmeter -g results.jtl -o report
```

The GUI is for **authoring and debugging only**. Every real measurement goes through headless mode — which also removes the single largest source of overhead.

APPLIES TO BOTH TOOLS

The universal rule is that **the load generator must never be saturated**. If its CPU, memory or network is maxed out, the target never received the intended load and no conclusion about the target is defensible. Publish the generator's own utilisation next to every result.

EXTENSIONS

Both ecosystems can add overhead — browser testing most of all

JMeter plugins

Simultaneously the greatest strength and the biggest operational risk. Listeners in particular are dangerous during high load; plugins also bring dependency and version-management burden.

k6 extensions

Overhead depends entirely on what the extension does. A data-generation extension is close to free. Modern k6 resolves supported extensions automatically; others need a custom binary.

THE EXPENSIVE CASE — REAL BROWSERS

k6 browser testing runs actual Chromium processes, which carry substantial per-VU cost. Grafana recommends a **hybrid model** rather than attempting thousands of full-browser users.

Do not do this

```
10,000 Chromium browser users
```

Do this instead

```
9,990 protocol-level users
+    10 real browser users
```

ADOPTION

Which is easier depends entirely on who is holding it

ROLE	BETTER FIT
Manual QA engineer	JMeter
QA who does not write code	JMeter
Java-heavy legacy team	JMeter
Developer	k6
SDET	k6
DevOps	k6
SRE	k6
JavaScript / TypeScript team	k6
CI/CD-first organisation	k6

One engineer, first test

JMeter feels easier: drop in a Thread Group, an HTTP Request, a Listener, press run. No syntax to learn.

Fifty engineers, three years

```
.js / .ts
  → Git
  → pull request
  → code review
  → CI
  → thresholds
```

That loop is far cleaner than repeatedly hand-editing large `.jmx` XML files that nobody can meaningfully review.

THE DISTINCTION THAT MATTERS

JMeter is easier to **start**. k6 is easier to **live with**. Softaims is optimising for the second.

CI/CD — THE DECISIVE FACTOR

Performance requirements become code that fails the build

```
export const options = {
  thresholds: {
    http_req_duration: [
      'p(95)<500',
      'p(99)<1000'
    ],
    http_req_failed: [
      'rate<0.01'
    ]
  }
};
```

The service-level objective lives beside the test, in version control, reviewed like any other change.

What happens on breach

```
CI pipeline
   ↓
k6 run
   ↓
threshold failed
   ↓
build fails
```

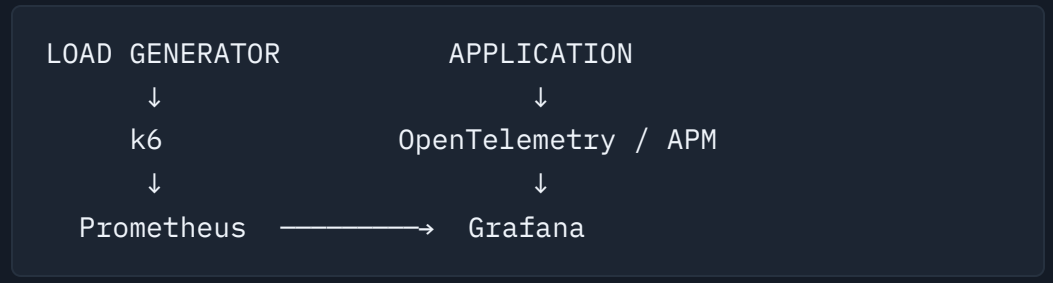
WHY THIS IS THE BIGGEST SINGLE FACTOR

A performance regression caught in CI costs minutes. The same regression found in production costs an incident. JMeter works in CI too — but k6's whole workflow already behaves like unit and integration testing, so nobody has to build the glue.

ANALYSIS

Do not stop at the tool's own summary report

The pipeline worth building



k6 supports metrics output integrations directly, which is a natural fit for a Grafana-oriented stack.

What to correlate

- RPS
- p95 / p99
- Error rate
- CPU
- Memory
- DB connections
- Cache hit rate
- Garbage collection
- Container throttling
- Queue depth

WHY CORRELATION IS THE WHOLE POINT

"p99 was 1.8 seconds" is an observation. **"p99 was 1.8 seconds, coinciding with database CPU at 95% and connection-pool saturation"** is a finding an engineer can act on. Only the second one changes anything.

LOAD SIZING

Little's Law turns a traffic figure into a test configuration

Concurrency ≈ Throughput × Response Time

1 · In-flight requests

```
Peak      = 500 RPS
p95       = 400 ms

500 × 0.4
= 200 in flight
```

2 · Transactions

```
4 API calls per
user transaction

500 ÷ 4
= 125 txn/sec

Transaction takes 2 s
125 × 2 = 250 VUs
```

3 · Add headroom

```
250 × 1.30
= 325 VUs

Never provision exactly
to the calculated figure.
```

The resulting k6 model

```
Target              125 iterations/sec
Requests/iteration    4
Expected RPS        ~500
Preallocated VUs    ~325
```

THE METRIC THAT PROTECTS YOU

k6 reports `dropped_iterations` when it cannot start work because no VU is free. That is the generator telling you it failed to deliver the workload — the single most important signal for trusting a result.

METHOD

The end-to-end process, whichever tool runs it

BEFORE TESTING

- **1 · Get production data.** Peak RPS, transaction mix, concurrency, payload sizes, think time — not “we need 1,000 users”.
- **2 · Define SLOs first.** p95 < 500 ms, p99 < 1 s, errors < 1%, CPU < 75%, pool < 80%.
- **3 · Model the business mix.** Login 10%, browse 50%, search 25%, checkout 10%, account 5%.
- **4 · Smoke test with one user.** Auth, correlation, tokens, cleanup, assertions.

RUNNING

- **5 · Baseline.** Low load to establish normal behaviour.
- **6 · Validate the generator.** Its CPU, RAM, network and dropped iterations — before trusting anything.
- **7 · Normal load.** Reproduce expected production peak.
- **8 · Stress.** Push past peak until behaviour changes materially.
- **9 · Spike.** Sudden events, e.g. 100 → 1,000 RPS, and measure recovery.
- **10 · Soak.** Sustained load long enough to expose leaks.

AFTER

- **11 · Correlate with telemetry.** A p99 figure means little until you can name the resource that caused it.
- **12 · Put small tests in CI.** Full stress and soak runs do not belong on every commit — smoke and baseline gates do.

> Steps 1, 2 and 6 are where most load-testing programmes fail — and none of them is about tool choice.

DIRECTION OF TRAVEL

Where the ecosystem is moving

INDICATOR	K6	JMETER
GitHub stars	31,368	9,523
GitHub forks	1,617	2,287
Current version	v2.2.0	5.6.3
Latest release date	Aug 2026	Jan 2024
Last commit	2 Sep 2026	27 Aug 2026
Cloud / CI focus	Very strong	Moderate
Legacy enterprise presence	Strong	Very strong

GitHub API, 2 September 2026. Forks favour JMeter, largely reflecting its greater age.

STAR RATIO

3.3×

in k6's favour — more answers when we are stuck, and more candidates who already know the tool

STATED FAIRLY

Both repositories are alive — JMeter had a commit last week. The meaningful difference is **delivery**: no stable JMeter release since January 2024, while k6 has moved through two major versions. That is not evidence JMeter is dying; it has an enormous installed base and unmatched plugin history.

WHERE JMETER IS SIMPLY BETTER

Protocol breadth is not a close contest

JMeter reaches

JDBC	SMTP
JMS	POP3 / IMAP
FTP	TCP
LDAP	Java objects
	SOAP-heavy systems

Built from the start around broad sampler and protocol support. If Softaims needs to load-test a database directly or drive a message queue, this is the tool — k6 cannot do it natively.

k6 reaches

HTTP / HTTPS	HTTP/2
REST APIs	gRPC
GraphQL	WebSocket
	Browser (Chromium)

Deliberately focused on modern application protocols, extensible beyond them. Official extensions now include SQL with MySQL and Postgres drivers — so “k6 cannot touch a database” is no longer strictly true, though JMeter remains far broader.

THIS DECISION WOULD FLIP

If Softaims' work were dominated by legacy enterprise infrastructure — direct JDBC load, JMS queues, LDAP directories — the recommendation on slide 2 would read the other way round. It does not, and that is why k6 wins here.

THE POLICY

What we standardise, workload by workload

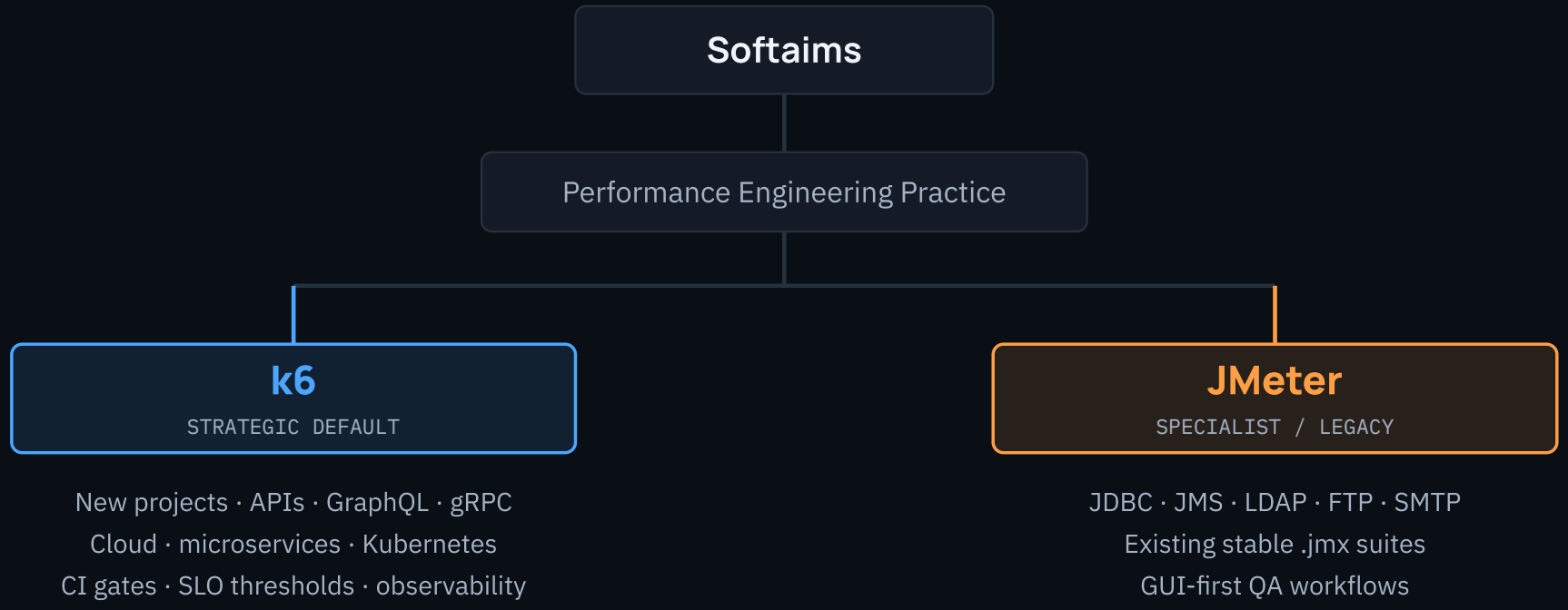
WORKLOAD	STANDARD
New REST / API performance test	k6
GraphQL API	k6
gRPC	k6
WebSocket	k6
CI performance gate	k6
Kubernetes / cloud-native	k6
Microservices	k6
Hybrid browser + backend	k6

WORKLOAD	STANDARD
Existing stable JMeter suite	Keep JMeter
JDBC direct load test	JMeter
JMS	JMeter
LDAP / FTP / SMTP	JMeter
GUI-only QA workflow	JMeter

> No migration project. Existing suites keep running until there is a reason to touch them.

TARGET STATE

Two platforms, one performance practice



BOTTOM LINE

Why k6, stated plainly

Not because JMeter's calculations are wrong — they are not. Because the combination is stronger for the work Softaims actually does.

k6 leads on

- Workload modelling
- CPU efficiency
- Memory efficiency
- CI/CD integration
- Git & code review
- Developer experience
- SLO automation
- Cloud & Kubernetes
- Modern API testing
- Release momentum

JMeter leads on

- Protocol breadth
- GUI friendliness for non-coders
- Legacy enterprise integration
- Version stability across years

THE DECISION

k6 = strategic default. **JMeter = specialist tool.**

This modernises our performance engineering without discarding the JMeter expertise and assets we already offer clients — while moving new work toward tests-as-code, CI gates, arrival-rate modelling and real observability.

REFERENCES

Sources

Primary — our own measurement

- **In-house benchmark**, 1,000 VUs, known 50 ms target. Harness and raw results: `benchmarks/` in the k6 Load Lab repository. Reproducible.
- **Load Testing Mathematics & Engineering** — internal 44-page reference, `benchmarks/`.

Official documentation

- Grafana k6 — open and closed models
  `grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/`
- Grafana k6 — built-in metrics reference
  `grafana.com/docs/k6/latest/using-k6/metrics/reference/`
- Grafana k6 — thresholds
  `grafana.com/docs/k6/latest/using-k6/thresholds/`
- Grafana k6 — arrival-rate VU allocation
  `grafana.com/docs/k6/latest/using-k6/scenarios/concepts/arrival-rate-vu-allocation/`
- Grafana k6 — browser hybrid approach
  `grafana.com/docs/k6/latest/using-k6-browser/recommended-practices/`

Official documentation, continued

- Grafana k6 — fine-tune OS, VU memory guidance
  `grafana.com/docs/k6/latest/set-up/fine-tune-os/`
- Grafana k6 — extensions
  `grafana.com/docs/k6/latest/extensions/`
- Apache JMeter — best practices (non-GUI mode, listeners, thread sizing, coordinated omission)
  `jmeter.apache.org/usermanual/best-practices.html`
- Apache JMeter — glossary (Elapsed Time, Latency)
  `jmeter.apache.org/usermanual/glossary.html`
- Apache JMeter — elements of a test plan
  `jmeter.apache.org/usermanual/test_plan.html`

Repository statistics & third-party

- GitHub API, retrieved 2 September 2026
  `api.github.com/repos/grafana/k6 · api.github.com/repos/apache/jmeter`
- Software Test Pilot — 10,000-VU k6 vs JMeter benchmark, July 2026. *Self-described as directional, not lab-grade; tested an older k6 release.*
  `softwaretestpilot.com/blog/performance-testing/`
- PeerSpot category mindshare, August 2026. *One site's engagement metric — not installed-base market share.*